บทที่ 2
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การพัฒนาระบบเว็บของ Chatshop ซึ่งรวมแชทจากหลายช่องทาง จัดการทีม แชทบอท สินค้า ลูกค้า คำสั่งซื้อ และรายงาน จำเป็นต้องศึกษาแนวคิดและเทคโนโลยีที่เกี่ยวข้อง เพื่อนำมาออกแบบและพัฒนาระบบให้ทำงานได้ตามวัตถุประสงค์ โดยมีหัวข้อดังนี้

2.1 LINE Messaging API

2.2 Meta Graph API

2.3 Webhook

2.4 Laravel

2.5 Blade และ React

2.6 Queue และ Task Scheduling

2.7 WebSocket และ Laravel Reverb

2.8 MySQL

2.9 MongoDB

2.10 สถาปัตยกรรม Multi-Tenant

2.11 การควบคุมสิทธิ์ตามบทบาท (RBAC)

2.12 OAuth 2.0

2.13 REST API

2.14 แชทบอท

2.15 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)

2.16 RAG (Retrieval-Augmented Generation)

2.17 Embedding และความคล้ายโคไซน์

2.18 Firebase Cloud Messaging (FCM)

2.19 งานวิจัยที่เกี่ยวข้อง

2.1 LINE Messaging API

LINE Messaging API คือบริการของ LINE สำหรับเชื่อมต่อระบบภายนอกกับบัญชี LINE Official Account ระบบรับข้อความจากผู้ใช้ผ่าน Webhook และส่งข้อความกลับผ่าน API โดยยืนยันตัวตนด้วย Channel Access Token และตรวจสอบ Webhook ด้วย Channel Secret

2.1.1 ข้อดี

2.1.1.1 เข้าถึงผู้ใช้ LINE ซึ่งเป็นแอปพลิเคชันแชทที่คนไทยใช้มากที่สุด

2.1.1.2 ส่งข้อความได้หลายรูปแบบ เช่น ข้อความ รูปภาพ และสติกเกอร์

2.1.2 ข้อเสีย

2.1.2.1 ไฟล์สื่อจากผู้ใช้ไม่มี URL สาธารณะ ต้องดาวน์โหลดผ่าน API

2.1.3 การนำไปใช้ในโครงการ ระบบรับข้อความจาก LINE ผ่าน Webhook ของแต่ละช่องทาง และส่งข้อความกลับด้วย Push API พร้อมแสดงสถานะกำลังโหลดระหว่างรอแชทบอท

2.2 Meta Graph API

Meta Graph API คือ API ของบริษัท Meta สำหรับเข้าถึงข้อมูลและบริการบน Facebook และ Instagram โครงงานนี้ใช้ส่วน Messenger Platform สำหรับรับส่งข้อความกับผู้ใช้ผ่านเพจ Facebook และบัญชี Instagram โดยใช้ Page Access Token ในการยืนยันตัวตน

2.2.1 ข้อดี

2.2.1.1 ใช้ API ชุดเดียวส่งข้อความได้ทั้ง Messenger และ Instagram

2.2.1.2 ดึงประวัติบทสนทนาเดิมของเพจมาเก็บในระบบได้

2.2.2 ข้อเสีย

2.2.2.1 ข้อความอัตโนมัติส่งได้ภายใน 24 ชั่วโมงหลังข้อความล่าสุดเท่านั้น

2.2.2.2 token ของเพจหมดอายุหรือถูกยกเลิกได้ ระบบต้องตรวจจับข้อผิดพลาดนี้ ปิดการเชื่อมต่อช่องทางนั้น และแจ้งให้ร้านค้าเชื่อมต่อใหม่

2.2.3 การนำไปใช้ในโครงการ ระบบใช้ Graph API เวอร์ชัน 19.0 ส่งข้อความตอบกลับ ดึงรูปโปรไฟล์ลูกค้า และนำเข้าประวัติแชทย้อนหลังได้ไม่เกิน 90 วัน

2.3 Webhook

Webhook คือกลไกที่ระบบต้นทางส่งคำขอ HTTP ไปยัง URL ที่ระบบปลายทางลงทะเบียนไว้ทันทีเมื่อเกิดเหตุการณ์ เช่น มีข้อความใหม่ ระบบปลายทางจึงรับข้อมูลได้ทันทีโดยไม่ต้องสอบถามซ้ำเป็นระยะ ผู้ให้บริการมักแนบลายเซ็นดิจิทัลที่สร้างจาก HMAC มาด้วย เพื่อให้ระบบปลายทางตรวจสอบได้ว่าข้อมูลมาจากผู้ให้บริการจริงและไม่ถูกแก้ไขระหว่างทาง

2.3.1 ข้อดี

2.3.1.1 รับข้อความใหม่ได้ทันที ไม่ต้องสอบถามข้อมูลซ้ำเป็นระยะ

2.3.1.2 ลดภาระของเครื่องแม่ข่ายทั้งฝั่งผู้ให้บริการและฝั่งระบบ

2.3.2 ข้อเสีย

2.3.2.1 ระบบต้องตอบกลับให้เร็ว หากตอบช้าผู้ให้บริการจะส่งซ้ำ

2.3.3 การนำไปใช้ในโครงการ ระบบตรวจสอบลายเซ็น X-Line-Signature ของ LINE และ X-Hub-Signature-256 ของ Meta ด้วย HMAC-SHA256 จากนั้นส่งข้อมูลเข้าคิวงานทันที เพื่อให้ตอบกลับผู้ให้บริการได้รวดเร็ว แล้วจึงบันทึกข้อความและเรียกแชทบอทในเบื้องหลัง

2.4 Laravel

Laravel คือเฟรมเวิร์กภาษา PHP ที่ใช้รูปแบบ MVC (Model-View-Controller) มีเครื่องมือพื้นฐานสำหรับพัฒนาระบบเว็บ ได้แก่ การกำหนดเส้นทาง Eloquent ORM สำหรับติดต่อฐานข้อมูล Blade สำหรับสร้างหน้าเว็บ และระบบคิวงานสำหรับประมวลผลเบื้องหลัง

2.4.1 ข้อดี

2.4.1.1 มีเครื่องมือพื้นฐานครบ ลดเวลาในการพัฒนา

2.4.1.2 โครงสร้างโค้ดเป็นระเบียบตามรูปแบบ MVC

2.4.1.3 มีเอกสารและชุมชนผู้ใช้งานจำนวนมาก

2.4.2 ข้อเสีย

2.4.2.1 ใช้ทรัพยากรเครื่องแม่ข่ายมากกว่าการเขียน PHP แบบไม่ใช้เฟรมเวิร์ก

2.4.2.2 ผู้พัฒนาต้องเรียนรู้ข้อกำหนดของเฟรมเวิร์กก่อนใช้งาน

2.4.3 การนำไปใช้ในโครงการ โครงงานนี้ใช้ Laravel 10 พัฒนาระบบเว็บทั้งหมด ทั้งหน้าเว็บ การรับ Webhook คิวงาน แชทบอท และ REST API ที่แอปพลิเคชันเรียกใช้

2.5 Blade และ React

Blade คือระบบ Template ของ Laravel สำหรับสร้างหน้าเว็บจากฝั่งเครื่องแม่ข่าย ส่วน React คือไลบรารี JavaScript สำหรับสร้างส่วนติดต่อผู้ใช้จากคอมโพเนนต์ เมื่อข้อมูลเปลี่ยน React จะปรับเฉพาะส่วนของหน้าจอที่เปลี่ยนแปลง จึงเหมาะกับหน้าจอที่มีข้อมูลใหม่เข้ามาตลอดเวลา

2.5.1 การนำไปใช้ในโครงการ หน้าเข้าสู่ระบบและหน้าตั้งค่าทั่วไปสร้างด้วย Blade ส่วนหน้ากล่องข้อความสร้างด้วย React เพื่อแสดงข้อความใหม่ได้ทันทีโดยไม่ต้องโหลดหน้าเว็บใหม่

2.6 Queue และ Task Scheduling

Queue คือการย้ายงานที่ใช้เวลานานออกไปทำงานเบื้องหลัง ทำให้ระบบตอบกลับคำขอได้ทันที งานแต่ละประเภทแยกไว้คนละคิวได้เพื่อกำหนดลำดับความสำคัญ ส่วน Task Scheduling คือการกำหนดให้คำสั่งทำงานอัตโนมัติตามรอบเวลา

2.6.1 การนำไปใช้ในโครงการ

2.6.1.1 คิว high ใช้ประมวลผล Webhook ขาเข้าและการนำเข้าประวัติแชท

2.6.1.2 คิว default ใช้สร้างคำตอบของแชทบอท สรุปบทสนทนา จัดทำดัชนีเอกสารความรู้ และส่งบรอดแคสต์เป็นชุด ชุดละ 100 บทสนทนา

2.6.1.3 คิว typing ใช้ส่งสถานะกำลังพิมพ์ระหว่างที่แชทบอทสร้างคำตอบ

2.6.1.4 ตัวจัดตารางเวลาตรวจหาบทสนทนาที่เกินเกณฑ์ SLA ทุก 1 นาที

2.7 WebSocket และ Laravel Reverb

WebSocket คือโพรโทคอลที่เปิดการเชื่อมต่อค้างไว้ระหว่างเบราว์เซอร์กับเครื่องแม่ข่าย ทำให้เครื่องแม่ข่ายส่งข้อมูลไปยังผู้ใช้ได้ทันที ส่วน Laravel Reverb คือเซิร์ฟเวอร์ WebSocket ของ Laravel ที่ใช้โพรโทคอลเดียวกับ Pusher จึงใช้ร่วมกับไลบรารี Laravel Echo ได้

2.7.1 การนำไปใช้ในโครงการ ระบบส่งเหตุการณ์ เช่น ข้อความใหม่ การแจ้งเตือน และคำสั่งซื้อที่เปลี่ยนสถานะ ผ่านช่องส่วนตัวของแต่ละร้านค้า หน้าเว็บจึงแสดงข้อมูลล่าสุดได้ทันที

2.8 MySQL

MySQL คือระบบจัดการฐานข้อมูลเชิงสัมพันธ์แบบโอเพนซอร์ส จัดเก็บข้อมูลในรูปตารางที่เชื่อมกันด้วยคีย์ ใช้ภาษา SQL ในการจัดการข้อมูล และรองรับธุรกรรมตามคุณสมบัติ ACID

2.8.1 ข้อดี

2.8.1.1 รักษาความถูกต้องของข้อมูลที่สัมพันธ์กันด้วยคีย์หลักและคีย์นอก

2.8.1.2 รองรับธุรกรรม เหมาะกับข้อมูลที่ต้องการความถูกต้องสูง

2.8.2 ข้อเสีย

2.8.2.1 ต้องกำหนดโครงสร้างตารางล่วงหน้า ไม่เหมาะกับข้อมูลที่เปลี่ยนบ่อย

2.8.3 การนำไปใช้ในโครงการ โครงงานนี้ใช้ MySQL จัดเก็บข้อมูลร้านค้า สมาชิก บทบาท ช่องทางที่เชื่อมต่อ สินค้า ลูกค้า คำสั่งซื้อ บรอดแคสต์ และการตั้งค่า

2.9 MongoDB

MongoDB คือฐานข้อมูลแบบเอกสาร จัดเก็บข้อมูลเป็นเอกสารรูปแบบ BSON ซึ่งมีโครงสร้างคล้าย JSON เอกสารในคอลเลกชันเดียวกันไม่จำเป็นต้องมีฟิลด์เหมือนกัน การใช้ฐานข้อมูลหลายประเภทในระบบเดียวตามลักษณะของข้อมูลเรียกว่า Polyglot Persistence (Fowler, 2011)

2.9.1 ข้อดี

2.9.1.1 โครงสร้างข้อมูลยืดหยุ่น เพิ่มฟิลด์ได้โดยไม่ต้องแก้โครงสร้างตาราง

2.9.1.2 รองรับข้อมูลปริมาณมากและเก็บอาร์เรย์ในเอกสารได้โดยตรง

2.9.2 ข้อเสีย

2.9.2.1 ไม่มีการบังคับความสัมพันธ์ระหว่างข้อมูลเหมือนฐานข้อมูลเชิงสัมพันธ์

2.9.3 การนำไปใช้ในโครงการ โครงงานนี้ใช้ MongoDB จัดเก็บบทสนทนาและข้อความจากทุกช่องทาง ซึ่งมีหลายประเภทและมีปริมาณมาก รวมถึงเอกสารความรู้ของแชทบอท

2.10 สถาปัตยกรรม Multi-Tenant

สถาปัตยกรรม Multi-Tenant คือการออกแบบให้ระบบเดียวให้บริการผู้เช่าหลายรายโดยข้อมูลของผู้เช่าแต่ละรายแยกออกจากกัน วิธีที่ใช้กันทั่วไปคือใช้ฐานข้อมูลร่วมกันและระบุรหัสผู้เช่าในทุกตาราง แล้วกรองข้อมูลตามรหัสผู้เช่าทุกครั้งที่อ่านหรือเขียน

2.10.1 การนำไปใช้ในโครงการ ทุกตารางและทุกเอกสารมีรหัสร้านค้ากำกับ ร้านค้าหนึ่งแห่งมีได้หลายธุรกิจ แต่ละธุรกิจมีช่องทาง แบรนด์ และการตั้งค่าแชทบอทของตนเอง

2.11 การควบคุมสิทธิ์ตามบทบาท (RBAC)

RBAC (Role-Based Access Control) คือการควบคุมการเข้าถึงระบบโดยกำหนดสิทธิ์ให้แก่บทบาท แล้วมอบบทบาทให้ผู้ใช้ เมื่อต้องการเปลี่ยนสิทธิ์ของผู้ใช้หลายคน ผู้ดูแลแก้ไขเฉพาะบทบาทโดยไม่ต้องแก้ทีละคน ช่วยลดความผิดพลาดในการกำหนดสิทธิ์และตรวจสอบสิทธิ์ของผู้ใช้ได้ง่าย

2.11.1 การนำไปใช้ในโครงการ ระบบมีบทบาทพื้นฐาน 5 บทบาท ได้แก่ owner, manager, admin, agent และ viewer และเจ้าของร้านสร้างบทบาทใหม่ได้ โดยเลือกจากสิทธิ์ทั้งหมด 12 รายการ เช่น การตอบข้อความ การมอบหมายงาน และการจัดการคำสั่งซื้อ

2.12 OAuth 2.0

OAuth 2.0 คือมาตรฐานการอนุญาตให้แอปพลิเคชันเข้าถึงข้อมูลของผู้ใช้บนบริการอื่นโดยผู้ใช้ไม่ต้องให้รหัสผ่านแก่แอปพลิเคชัน ผู้ใช้เข้าสู่ระบบและยินยอมที่ผู้ให้บริการ แล้วแอปพลิเคชันจะได้รับ Access Token สำหรับเรียกใช้ API แทนผู้ใช้

2.12.1 การนำไปใช้ในโครงการ ระบบใช้ Laravel Socialite ให้ผู้ใช้เข้าสู่ระบบด้วยบัญชี Facebook หรือ Google และใช้ OAuth ของ Facebook ขอสิทธิ์จัดการข้อความของเพจ

2.13 REST API

REST API คือรูปแบบการออกแบบส่วนต่อประสานสำหรับสื่อสารระหว่างระบบผ่านโพรโทคอล HTTP โดยระบุทรัพยากรด้วย URL และใช้เมธอด GET POST PUT PATCH และ DELETE แทนการอ่าน สร้าง และแก้ไขข้อมูล ข้อมูลที่รับส่งนิยมใช้รูปแบบ JSON

2.13.1 การนำไปใช้ในโครงการ ระบบเปิด REST API ประมาณ 99 เส้นทางภายใต้ mobile/ ให้แอปพลิเคชันบนสมาร์ตโฟนเรียกใช้ โดยยืนยันตัวตนด้วยโทเคนของ Laravel Sanctum

2.14 แชทบอท

แชทบอทคือโปรแกรมที่สนทนากับผู้ใช้ด้วยภาษาธรรมชาติผ่านช่องทางแชท แชทบอทแบบกฎจะตอบตามคำสำคัญหรือเงื่อนไขที่กำหนดไว้ล่วงหน้า ส่วนแชทบอทที่ใช้แบบจำลองภาษาสามารถสร้างคำตอบใหม่ได้ตามบริบทของการสนทนา แต่อาจสร้างข้อมูลที่ไม่มีอยู่จริงได้ ระบบแชทบอทสำหรับร้านค้าจึงต้องมีกลไกส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อคำถามอยู่นอกเหนือข้อมูลที่แชทบอทมี

2.14.1 การนำไปใช้ในโครงการ แชทบอทตอบลูกค้าได้ทุกช่องทาง โดยร้านค้ากำหนดโหมดเริ่มต้นของแต่ละธุรกิจ และเปลี่ยนโหมดแยกตามบทสนทนาได้ 3 โหมด ดังนี้

2.14.1.1 Manual เจ้าหน้าที่ตอบลูกค้าเองทั้งหมด

2.14.1.2 Suggest แชทบอทเสนอคำตอบให้เจ้าหน้าที่เลือกใช้

2.14.1.3 Autopilot แชทบอทตอบลูกค้าเอง ส่งต่อให้เจ้าหน้าที่เมื่อตอบไม่ได้

2.14.2 การตอบอัตโนมัติด้วยคีย์เวิร์ด นอกจากแชทบอทที่ใช้แบบจำลองภาษา ระบบมีการตอบอัตโนมัติแบบกฎ ร้านค้ากำหนดคีย์เวิร์ดและข้อความตอบกลับได้เอง เมื่อข้อความของลูกค้าตรงกับคีย์เวิร์ดทั้งข้อความหรือมีคีย์เวิร์ดอยู่ ระบบจะส่งข้อความที่กำหนดไว้ทันที

2.15 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)

แบบจำลองภาษาขนาดใหญ่คือแบบจำลองที่ผ่านการฝึกด้วยข้อความจำนวนมหาศาลเพื่อทำนายคำถัดไปของข้อความ แบบจำลองส่วนใหญ่ใช้สถาปัตยกรรม Transformer ซึ่งใช้กลไก Self-Attention พิจารณาความสัมพันธ์ระหว่างคำทุกคำในข้อความพร้อมกัน (Vaswani et al., 2017) เมื่อแบบจำลองมีขนาดใหญ่เพียงพอ จะสามารถทำงานใหม่ได้จากคำสั่งและตัวอย่างที่ให้ไว้ในข้อความนำเข้า โดยไม่ต้องฝึกแบบจำลองเพิ่มเติม (Brown et al., 2020)

2.15.1 การนำไปใช้ในโครงการ ระบบประกอบข้อความสั่งงานจากบุคลิกที่ร้านค้ากำหนด ข้อมูลแบรนด์ ผลการค้นคืนจากฐานความรู้ บทสรุปของบทสนทนา และข้อความล่าสุด 20 ข้อความ แล้วส่งให้แบบจำลองภาษาสร้างคำตอบ ร้านค้าเลือกใช้ Ollama ที่ติดตั้งบนเครื่องแม่ข่ายเอง หรือบริการผ่าน API เช่น Anthropic Claude และ Google Gemini ได้

2.16 RAG (Retrieval-Augmented Generation)

RAG คือเทคนิคที่ค้นคืนเอกสารที่เกี่ยวข้องกับคำถามจากฐานความรู้ก่อน แล้วส่งเอกสารเหล่านั้นให้แบบจำลองภาษาใช้ประกอบการสร้างคำตอบ (Lewis et al., 2020) ช่วยให้แบบจำลองตอบจากข้อมูลของร้านค้าที่ไม่ได้อยู่ในข้อมูลที่ใช้ฝึก และลดการสร้างข้อมูลที่ไม่มีอยู่จริง

2.16.1 การนำไปใช้ในโครงการ ร้านค้าเพิ่มเอกสารความรู้และนำข้อมูลสินค้าเข้าสู่ฐานความรู้ได้ ระบบแบ่งเอกสารเป็นส่วนย่อยขนาดไม่เกิน 500 ตัวอักษร จัดเก็บพร้อมเวกเตอร์ใน MongoDB และค้นคืนส่วนย่อยที่เกี่ยวข้อง 4 อันดับแรกมาประกอบข้อความสั่งงาน

2.17 Embedding และความคล้ายโคไซน์

Embedding คือการแปลงข้อความเป็นเวกเตอร์ตัวเลขที่มีมิติคงที่ โดยข้อความที่มีความหมายใกล้เคียงกันจะได้เวกเตอร์ที่อยู่ใกล้กันในปริภูมิเวกเตอร์ (Mikolov et al., 2013) ความใกล้เคียงวัดได้จากความคล้ายโคไซน์ $\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$ ค่าที่เข้าใกล้ 1 แสดงว่าข้อความทั้งสองมีความหมายใกล้เคียงกัน ส่วนค่าที่เข้าใกล้ 0 แสดงว่าไม่เกี่ยวข้องกัน

2.17.1 การนำไปใช้ในโครงการ ระบบแปลงเอกสารและคำถามเป็นเวกเตอร์ด้วยแบบจำลอง Embedding ของ Google Gemini หรือ Ollama แล้วคำนวณความคล้ายโคไซน์ระหว่างคำถามกับทุกส่วนย่อย และเลือกเฉพาะส่วนย่อยที่มีค่าตั้งแต่ 0.3 ขึ้นไป

2.18 Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging คือบริการส่งการแจ้งเตือนแบบพุชของ Google เครื่องแม่ข่ายส่งการแจ้งเตือนไปยังอุปกรณ์ผ่านโทเคนประจำเครื่องที่แอปพลิเคชันลงทะเบียนไว้ ผู้ใช้จึงได้รับการแจ้งเตือนแม้ไม่ได้เปิดแอปพลิเคชันอยู่ และแตะการแจ้งเตือนเพื่อเปิดหน้าจอที่เกี่ยวข้องได้

2.18.1 การนำไปใช้ในโครงการ ระบบเว็บส่งการแจ้งเตือนไปยังแอปพลิเคชันของเจ้าหน้าที่เมื่อมีข้อความใหม่ บทสนทนาค้างตอบเกินเกณฑ์ SLA หรือแชทบอทไม่สามารถสร้างคำตอบได้

2.19 งานวิจัยที่เกี่ยวข้อง

[รอข้อมูล: งานวิจัยหรือโครงงานที่เกี่ยวข้อง 3–5 เรื่อง พร้อมชื่อผู้แต่ง ปี และสรุปสั้นๆ]